Chapter 1: Introduction to Python and Variables

Python is a high-level, interpreted programming language created by Guido van Rossum and first released in 1991. It is known for readable syntax, which uses indentation instead of curly braces to define blocks of code. Python is used in web development, data science, automation and artificial intelligence.

A variable is a name that refers to a value stored in memory. Python is dynamically typed, so you do not declare the type of a variable; the type is decided at run time from the assigned value. The four basic built-in types are int (whole numbers), float (decimal numbers), str (text) and bool (True or False). The built-in function type() tells you the type of any value.

Variable names may contain letters, digits and underscores, but they cannot start with a digit and cannot be a reserved keyword such as if or class. By convention, Python programmers use snake_case for variable names, for example student_name.

```
age = 20
name = 'Ali'
height = 5.9
is_student = True
print(type(age))   # <class 'int'>
```

Chapter 2: Control Flow: Conditions and Loops

Control flow statements decide which lines of code run. The if statement runs a block only when its condition is True. You can add elif branches for extra conditions and an else branch that runs when none of the conditions match.

A for loop repeats a block once for every item in a sequence such as a list, a string or a range. The range(n) function produces the numbers 0 to n-1. A while loop repeats as long as its condition stays True, so you must make sure the condition eventually becomes False, otherwise you create an infinite loop.

Inside any loop, the break statement stops the loop immediately, while the continue statement skips the rest of the current iteration and moves to the next one. Loops can be nested, but deeply nested loops are harder to read and slower to run.

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Chapter 3: Functions

A function is a reusable block of code defined with the def keyword. Functions take parameters as input and send a result back using the return statement. If a function has no return statement, it returns None.

Parameters can have default values, which are used when the caller does not supply an argument. A function can accept any number of positional arguments with *args and any number of keyword arguments with **kwargs. A lambda is a small anonymous function written in one line, for example lambda x: x * 2.

Variable scope in Python follows the LEGB rule: Python looks for a name first in the Local scope, then the Enclosing function scope, then the Global scope and finally the Built-in scope. A variable created inside a function is local and disappears when the function ends, unless it is declared with the global keyword.

```
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}!'

print(greet('Sara'))
```

Chapter 4: Lists, Tuples, Dictionaries and Sets

A list is an ordered, mutable collection written with square brackets. You can add items with append(), remove them with remove() or pop(), and change items by index. A tuple is like a list but immutable, which means it cannot be changed after creation; it is written with round brackets.

A dictionary stores key-value pairs inside curly braces and gives very fast lookup by key. Keys must be unique and immutable. A set is an unordered collection of unique items and supports mathematical operations such as union, intersection and difference.

A list comprehension is a short way to build a new list from an existing sequence. For example, [x * x for x in range(5)] creates the list of the first five square numbers. Choosing the right collection matters: use a list for ordered data, a tuple for fixed data, a dictionary for lookups and a set for removing duplicates.

```
marks = {'math': 90, 'physics': 85}
print(marks['math'])
squares = [x * x for x in range(5)]
```

Chapter 5: File Handling and Exceptions

Python reads and writes files with the open() function. The mode argument controls what you can do: 'r' reads, 'w' writes and overwrites the file, and 'a' appends to the end. The recommended way to open a file is the with statement, because it closes the file automatically even if an error occurs.

An exception is an error that happens while the program is running, such as dividing by zero or opening a file that does not exist. You handle exceptions with try and except blocks. Code in the try block is executed first; if an exception occurs, the matching except block runs. The optional finally block always runs, whether or not an error happened.

You can raise your own exception with the raise keyword, for example raise ValueError('Age cannot be negative'). Catching specific exception types, such as FileNotFoundError, is better than a bare except, because a bare except can hide real bugs.

```
try:
    with open('data.txt') as f:
        print(f.read())
except FileNotFoundError:
    print('File not found')
```

Chapter 6: Object-Oriented Programming

Object-oriented programming (OOP) organises code into classes and objects. A class is a blueprint, and an object is an instance created from that blueprint. The special method __init__ runs when an object is created and is used to set up its attributes. The first parameter of every method is self, which refers to the current object.

Inheritance lets a child class reuse the attributes and methods of a parent class, written as class Child(Parent). Encapsulation means keeping data and the methods that use it together, and hiding internal details; by convention a name starting with an underscore is treated as private.

Polymorphism means that different classes can provide their own version of the same method name, so the same call behaves differently depending on the object. For example, both Dog and Cat classes may define speak(), and each returns a different sound.

```
class Student:
    def __init__(self, name):
        self.name = name
    def intro(self):
        return f'I am {self.name}'
```